

Chapter 7 - Object-Oriented Programming 3

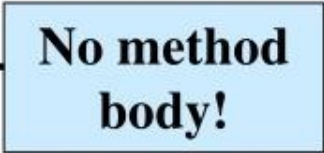
Abstract Class

- There are some situations in which it is useful to define base classes that are never instantiated.
- Such classes are called *abstract* classes.
- For instance, we could have an abstract superclass *Shape* and derive concrete classes (non-abstract classes) such as *Square*, *Circle*, *Triangle* etc.
- We do not know how to implement the `area()` method of the *Shape* class - leave it empty and let subclasses to define it (*abstract method*).

Abstract Methods

- A method declared as *abstract* is **NOT** allowed to contain a body.

```
public abstract void f() ;
```



No method
body!



- To force the subclasses to define the method.
- A class **must** be declared as **abstract** if it contains one or more abstract methods.

Abstract Class Syntax

- A class is made abstract by declaring it with the keyword `abstract`.

The diagram illustrates the syntax of an abstract class. It shows a code snippet with two callout boxes. The first box, labeled 'Abstract Class', points to the word 'abstract' in the class declaration. The second box, labeled 'Abstract Method', points to the word 'abstract' in the method signature.

```
public abstract class Shape {  
    String name;  
    public abstract double area();  
    public String name() {  
        return name;  
    }  
  
    //other stuff  
}
```

- An *abstract* class cannot be instantiated.

Can't Instantiate an Abstract Class

- The following example tries to instantiate an abstract class.

```
abstract class Shape {  
    String name;  
    abstract public double area();  
    public String name() {  
        return name;  
    }  
  
    //other stuff  
}  
class TestShape {  
    public static void main(String[] args) {  
        Shape s = new Shape();  
    }  
}
```

```
>javac TestShape.java
```

```
TestShape.java:10: Shape is abstract; cannot be instantiated
```

```
    Shape s = new Shape();
```

```
        ^
```

```
1 error
```

Abstract Method => Abstract Class

- if a class contains even one abstract method, then the class itself has to be declared to be abstract.

```
class Shape {
    String name;
    abstract public double area();
    public String name() { return name; }

    //other stuff
}
class TestShape {
    public static void main(String[] args) {
        Shape s = new Shape();
    }
}
```

```
>javac TestShape2.java
TestShape2.java:1: Shape should be declared abstract; it does not
define area() in Shape
class Shape {
^
1 error
```


Example

- The class inherits from **Shape** MUST implement (provide a method body) **area()**.

```
abstract class Shape {
    String name;
    abstract public double area();
    public String name() {
        return name;
    }

    //other stuff
}

class Square extends Shape {
    double length;
    public Square(double length) {
        this.length = length;
    }
    public double area() {
        return length*length;
    }
}

class TestShape3 {
    public static void main(String[] args) {
        Square s = new Square(12.5);
        System.out.println("The area of square s is "
            + s.area());
    }
}
```

Output

The area of square s is 156.25

Introduction to Polymorphism

- Polymorphism
 - Polymorphism means "having many forms."
 - In Java, it means that a single variable might be used with several different types of related objects at different times in a program.
 - Recall that a superclass reference can refer to subclass objects.
 - When the variable is used with "dot notation" `variable.method()` to invoke a method, exactly which method is run depends on the object that the variable currently refers to.

Example

- Consider the previous example, suppose we want to add a new class **Circle** that inherits from **Shape**.

// Shape and Square are the same as before

```
class Circle extends Shape {  
    double radius;  
    public Circle(double radius) {  
        this.radius = radius;  
    }  
    public double area() {  
        return Math.PI*radius*radius;  
    }  
}
```

```
class TestShape4 {  
    public static void main(String[] args) {  
        Shape s = new Square(12.5);  
        System.out.println("The area of s is " + s.area());  
  
        s = new Circle(5);  
        System.out.println("The area of s is " + s.area());  
    }  
}
```

Output

```
The area of s is 156.25  
The area of s is 78.539816339744
```

s.area()
calls the
correct
method

Shape **s** can
refer to a
Square or a
Circle.

Advantages of using Polymorphism

- Polymorphism
 - Reduce amount of work in distinguishing and handling object types
 - Old approach - Determine the object type and then uses a nested if-else statement to execute the appropriate statements (difficult and easy to make mistake)
 - Helps build extensible systems
 - Program can be written to process objects of types that may not exist when the program is under development.
 - Can add classes to systems easily

Polymorphism Examples

- Video game
 - Superclass **GamePiece**
 - Contains method **drawYourself**
 - Subclasses **Martian**, **Venutian**, **LaserBeam**, etc.
 - Override method **drawYourself**
 - **Martian** draws itself with antennae
 - **LaserBeam** draws itself as bright red beam
 - This is polymorphism
 - Easily extensible
 - Suppose we add class **Mercurian**
 - Class **Mercurian** inherits superclass **GamePiece**
 - Overrides method **drawYourself**

Case Study: A Payroll System Using Polymorphism

- Example
 - Abstract superclass **Employee**
 - Method **earnings** applies to all employees
 - Person's earnings dependent on type of **Employee**
 - Concrete **Employee** subclasses declared **final**
 - **Boss**
 - **CommissionWorker**
 - **HourlyWorker**

```
abstract class Employee {
    private String firstName;
    private String lastName;

    // Constructor
    public Employee( String first, String last ) {
        firstName = new String ( first );
        lastName = new String( last );
    }

    // Return a copy of the first name
    public String getFirstName() {
        return new String( firstName );
    }

    // Return a copy of the last name
    public String getLastName() {
        return new String( lastName );
    }

    /* Abstract method that must be implemented for each
     * derived class of Employee from which objects are
     * instantiated.
     */
    abstract double earnings();
}
```

```
public final class Boss extends Employee {
    private double weeklySalary;

    // Constructor for class Boss
    public Boss( String first, String last, double s) {
        super( first, last ); // call base-class constructor
        setWeeklySalary( s );
    }

    // Set the Boss's salary
    public void setWeeklySalary( double s ) {
        weeklySalary = (s > 0) ? s : 0;
    }

    // Get the Boss's pay
    public double earnings() {
        return weeklySalary;
    }

    // Print the Boss's name
    public String toString() {
        return "Boss: " + getFirstName() + " "
            + getLastName();
    }
}
```




```
// CommissionWorker class derived from Employee
final class CommissionWorker extends Employee {
    private double salary;        // base salary per week
    private double commission;    // amount per item sold
    private int quantity;        // total items sold for week

    // Constructor for class CommissionWorker
    public CommissionWorker( String first, String last, double s,
                           double c, int q) {
        super( first, last );    // call base-class constructor
        setSalary( s );
        setCommission( c );
        setQuantity( q );
    }

    // Set CommissionWorker's weekly base salary
    public void setSalary( double s ) { salary = (s>0) ? s : 0; }

    // Set CommissionWorker's commission
    public void setCommission(double c) {commission = (c>0) ? c : 0;}

    // Set CommissionWorker's quantity sold
    public void setQuantity( int q ) { quantity = (q>0) ? q : 0; }

    // Determine CommissionWorker's earnings
    public double earnings(){ return salary + commission * quantity; }

    // Print the CommissionWorker's name
    public String toString(){
        return "Commission worker: "+getFirstName()+" "+getLastName();
    }
}
```

```
// Definition of class HourlyWorker
final class HourlyWorker extends Employee {
    private double wage;    // wage per hour
    private double hours;   // hours worked for week

    // Constructor for class HourlyWorker
    public HourlyWorker( String first, String last, double w,
                        double h ) {
        super( first, last );    // call base-class constructor
        setWage( w );
        setHours( h );
    }

    // Set the wage
    public void setWage( double w ) { wage = (w>0) ? w : 0; }

    // Set the hours worked
    public void setHours( double h ) {
        hours = (h>=0 && h<168) ? h : 0;
    }

    // Get the HourlyWorker's pay
    public double earnings() { return wage * hours; }

    public String toString() {
        return "Hourly worker: " + getFirstName() + " "+getLastName();
    }
}
```

```
// Driver for Employee hierarchy
class TestEmployee1 {
    public static void main(String[] args) {

        Employee emp; // base-class reference
        emp = new Boss( "John", "Smith", 800.00 );
        System.out.println(emp.toString() + " earned $"
                           + emp.earnings() );

        emp = new CommissionWorker( "Sue", "Jones", 400.0, 3.0,
                                     150);
        System.out.println(emp.toString() + " earned $"
                           + emp.earnings() );

        emp = new HourlyWorker( "Karen", "Price", 13.75, 40 );
        System.out.println(emp.toString() + " earned $"
                           + emp.earnings() );
    }
}
```

Output

```
Boss: John Smith earned $800.0
Commission worker: Sue Jones earned $850.0
Hourly worker: Karen Price earned $550.0
```

Example (Continued)

- The above test program can be implemented by using an **Employee** array.

```
class TestEmployee2 {  
    public static void main(String[] args) {  
  
        Employee[] emps = new Employee[3]; // base-class references  
  
        emps[0] = new Boss( "John", "Smith", 800.00 );  
        emps[1] = new CommissionWorker( "Sue", "Jones", 400.0, 3.0,  
                                         150 );  
        emps[2] = new HourlyWorker( "Karen", "Price", 13.75, 40 );  
  
        for (int i=0; i< emps.length; i++) {  
            System.out.println(emps[i] + " earned $"  
                               + emps[i].earnings() );  
        }  
    }  
}
```

toString() will be
called automatically.

Example - Add a New Worker

- Suppose we have to add a new worker of a new **PieceWorker** class.

```
final class PieceWorker extends Employee {
    private double wagePerPiece; // wage per piece output
    private int quantity;        // output for week

    // constructor for class PieceWorker
    public PieceWorker(String first, String last, double wage,
                       int num) {
        super( first, last ); // call superclass constructor
        setWage( wage );
        setQuantity( num );
    }

    // set PieceWorker's wage
    public void setWage(double wage) { wagePerPiece=(wage>0)?wage:0; }

    // set number of items output
    public void setQuantity( int numberOfItems ) {
        quantity = (numberOfItems > 0) ? numberOfItems : 0;
    }

    // determine PieceWorker's earnings
    public double earnings() { return quantity * wagePerPiece; }
    public String toString() {
        return "Piece worker: " + getFirstName() + " " + getLastName();
    }
} // end class PieceWorker
```


- Let's how easy we can modify the main program.

```
// Driver for Employee hierarchy
class TestEmployee3 {
    public static void main(String[] args) {

        Employee[] emps = new Employee[4]; // base-class reference

        emps[0] = new Boss( "John", "Smith", 800.00 );
        emps[1] = new CommissionWorker( "Sue", "Jones", 400.0, 3.0,
                                         150);
        emps[2] = new HourlyWorker( "Karen", "Price", 13.75, 40 );
        emps[3] = new PieceWorker( "Maggie", "Jackson", 5.5, 200 );

        for (int i=0; i< emps.length; i++) {
            System.out.println(emps[i] + " earned $"
                               + emps[i].earnings() );
        }
    }
}
```

Output

```
>java TestEmployee3
Boss: John Smith earned $800.0
Commission worker: Sue Jones earned $850.0
Hourly worker: Karen Price earned $550.0
Piece worker: Maggie Jackson earned $1100.0
```

Interface

- Java does not support multiple inheritance: a class cannot have more than one superclass.
- However, Java provides a concept similar to the class, called **interface** to enable multiple "inheritance".
- An interface is a collection of abstract methods.
- With an interface, all methods are implicitly public and abstract, and all data members are implicitly final, public and static. (i.e. they are constants).

Interface Example

```
Public class TV {  
}  
  
public interface Recordable {  
    void Record();  
    void Play();  
}  
  
public class TV_VCR extends TV implements  
    Recordable {  
    public void Record() {  
    }  
  
    public void Play() {  
    }  
  
    //.....  
}
```

Interface Example

- Since the methods of an interface are implicitly **public**, the overriding methods of the implementing class **MUST** be declared **public**.
- The overriding methods of the implementing class **MUST** have bodies or the implementing class **MUST** be declared abstract.